

O3LS: Optimizing Lattice Surgery via Automatic Layout Searching and Loose Scheduling

1st Chenghong Zhu*

*The Hong Kong University of
Science and Technology (Guangzhou)*
Guangzhou, China

2nd Xian Wu*

*The Hong Kong University of Science
and Technology (Guangzhou)*
Guangzhou, China

3rd Jiahan Chen

*The Hong Kong University of
Science and Technology (Guangzhou)*
Guangzhou, China

4th Keming He

*The Hong Kong University of
Science and Technology (Guangzhou)*
Guangzhou, China

5th Junjie Wu

*College of Computer Science and Technology
National University of Defense Technology*
Changsha, China

6th Xin Wang[†]

*The Hong Kong University of
Science and Technology (Guangzhou)*
Guangzhou, China
felixxinwang@hkust-gz.edu.cn

7th Lingling Lao[†]

*College of Computer Science and Technology
National University of Defense Technology*
Changsha, China
laolinglingrolls@gmail.com

Abstract—Toward the large-scale, practical realization of quantum computing, quantum error correction is essential. Among various quantum error-correcting codes, the surface code stands out as a leading candidate, and lattice surgery based on surface codes has emerged as a promising technique for fault-tolerant quantum computation (FTQC). However, implementing quantum algorithms using lattice surgery introduces both resource and time overhead. Existing approaches typically focus on large layout designs, with compiler passes aimed primarily at optimizing time overhead. This often overlooks the trade-off between rotation bottlenecks and movement distance, which leads to inefficient resource utilization and prevents further reduction of the quantum computation failure rate.

To address these challenges, we introduce O3LS, a framework for optimizing lattice surgery through automatic layout search and loose scheduling. O3LS achieves an optimal balance by automatically generating squeezed data layouts to reduce space requirements and employing loose scheduling algorithms combined with circuit synthesis techniques to reduce time overhead, thereby effectively minimizing overall logical error rates. Numerical results indicate that O3LS can reduce space overhead by 28.0% over standard layouts and 46.7% over sparse layouts without increasing the number of time steps, leading to suppression of logical error rates by up to 16% relative to larger data layout designs. O3LS can also achieve time overhead reductions of 36.07% and 24.76% in compact and standard data layout designs,

respectively. It suppresses logical error rates by up to an order of magnitude compared to prior compilers that focus primarily on maximizing parallelism.

Index Terms—fault-tolerant quantum computation, surface code, lattice surgery, quantum compiler design

I. INTRODUCTION

Quantum computers are expected to offer practical advantages over classical computers in solving certain classes of problems [22], [44]. While noisy intermediate-scale quantum (NISQ) devices [40] have enabled significant theoretical and experimental progress [3], [10], [15], [36], their performance remains limited by noise, hindering practical quantum advantage. This necessitates the use of quantum error correction (QEC) [6], [17], [19], [43], [55], which is critical for enabling fault-tolerant quantum computing (FTQC).

A series of experimental demonstrations have validated the feasibility of QEC [9], [16], [29], [37], [50], [58]. Among the various QEC codes, the surface code has emerged as a leading candidate for realizing FTQC, owing to its relatively high error threshold, compatibility with nearest-neighbor qubit connectivity and various schemes for universal quantum computation. Notably, recent implementations on Google's Willow processor have demonstrated exponential noise suppression using the surface code [1], [2], representing a significant milestone toward practical FTQC.

A prominent technique for achieving universal quantum computation on 2D nearest-neighbor devices is lattice surgery, which implements logical multi-qubit gates by merging and splitting planar code patches [26], [34]. This approach enables the execution of logical operations within a 2D nearest-

*Co-first authors

[†]Co-corresponding authors

This work has been partially supported by the National Key R&D Program of China (Grant No. 2024YFB4504001), the National Natural Science Foundation of China (Grant Nos. 12447107, 62302395, and 62421002), the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (Grant No. JYB2025XDXM202), the Aid Program for Science and Technology Innovative Research Teams in Higher Educational Institutions of Hunan Province, the CCF-Tencent Rhino-Bird Open Research Fund, and the Guangdong Provincial Quantum Science Strategic Initiative (Grant Nos. GDZX2403008 and GDZX2503001).

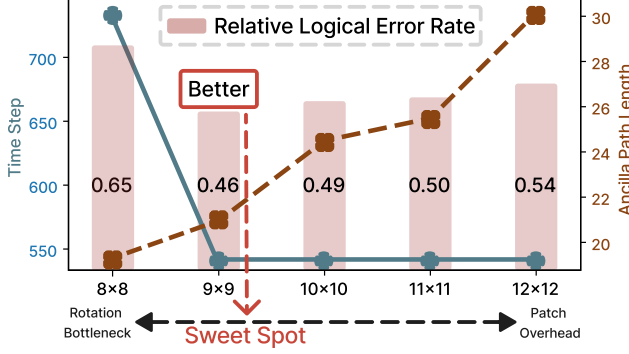


Fig. 1. O3LS can achieve comparable time overheads by automatically designing squeezed data layouts, even compared to larger layouts (e.g. 10×10 and 12×12). Moreover, squeezed layouts can reduce ancilla patch distances, thereby leading to a lower logical error rate (sweet spot).

neighbor layout and introduces additional compilation challenges, particularly in ancilla routing and the design of data layouts, with the ultimate goal of minimizing the logical error rate (LER).

Existing compilers [25], [47], [49] for lattice surgery compilation primarily focus on large or sparse data layouts to maximize parallelism. This strategy aims to reduce the total number of time steps required for executing quantum circuits, thereby minimizing LER. However, it overlooks a critical factor: **larger data layouts often result in increased movement distances**, which in turn elevate the likelihood of idle memory errors and ultimately degrade LER [28]. In contrast, compact data layouts can reduce the physical qubit footprint and limit movement overhead. Nevertheless, **if the layout architecture becomes too compact, it may significantly increase time costs** due to reduced parallelism. This reveals **a fundamental trade-off between the number of time steps and the movement distance** as illustrated in Fig. 1, which must be carefully balanced to optimize overall LER.

Motivated by these insights, our work leverages data layout design (sweet spot in Fig. 1) to reduce overall space overhead, while incorporating an optimized scheduling strategy to minimize time costs. Together, these approaches aim to achieve a more balanced and efficient reduction in the overall LER. In summary, our key contributions are:

- We develop a lattice surgery compiler O3LS that integrates an automatic data layout design algorithm, an optimized synthesis algorithm, and a loose scheduling strategy to minimize both the space and time overhead.
- We propose an automatic logical qubit data layout design strategy aimed for squeezing data layouts, enabling more efficient use of limited qubit resources to support larger quantum applications.
- We design an optimized synthesis algorithm that improves Pauli operator cancellation, thereby reducing the time overhead of lattice surgery in more compact data layout designs and making it well-suited for integration with the data layout design algorithm.

- We also present a loose scheduling mechanism that dynamically reassigns patch functionalities. This approach reduces redundant patch movements and eliminates unnecessary operations.

The evaluation results demonstrate that O3LS simultaneously achieves time step reductions of 36.07% and 24.76%, and space overhead reductions of 28.0% and 46.7% on average, compared to previous compilers executed on fixed layouts. These improvements contribute to an effective suppression of overall logical error rates.

II. BACKGROUND AND MOTIVATION

A. Quantum Computing and Quantum Error Correction

Introduction to Quantum Computation. In quantum computing, the basic unit of quantum information is qubit, which has two basis states, typically denoted as $|0\rangle$ and $|1\rangle$. A qubit can exist in a superposition expressed as $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, where $\alpha, \beta \in \mathbb{C}$ and $|\alpha|^2 + |\beta|^2 = 1$. Quantum gates serve as fundamental operations for manipulating qubit states.

Quantum Error Correction. Quantum computers are vulnerable to noise that disturbs quantum states. Quantum error correction codes (QECCs) address this by encoding a logical qubit into multiple physical qubits. Among these, the surface code [17] is particularly notable for its topological structure and compatibility with universal quantum operations. In Fig. 2(a), we show an example of our FTQC scheme using a surface code tile. Each tile has four boundaries, which are either of X or Z type, indicated by white and red lines. The logical Z operator is defined as the tensor product of physical Z operators along a string of data qubits, as illustrated by the red dashed lines. The logical X operator is defined similarly using physical X operators.

Common Gates and Their Decomposition. The universal gate set Clifford+T is well-suited for surface codes. It consists of the Hadamard gate H , the phase gate S , the T gate requiring the consumption of magic state $|0\rangle + e^{i\pi/4}|1\rangle$, and the controlled-NOT gate (CNOT). However, implementing T gates requires the consumption of magic states, which must be generated through magic state distillation protocols [7], [8], [27], [35] to enable universal quantum computation.

B. Lattice Surgery in Surface Code

Abstraction of surface code to patches. Each patch corresponds to a distance- d surface code, which encodes a logical qubit using d^2 physical data qubits, as illustrated in Fig. 2(b) for the case of $d = 3$. These surface codes are then abstracted as patches placed on tiles, as shown in Fig. 2(c). In this abstraction, dashed boundaries represent Z operators, while solid boundaries represent X operators. These operators can be represented as edges on the patch and, for convenience, will be referred to as X- and Z-edges. The key performance metric is the implementation of quantum algorithms using the minimum number of tiles and time steps—collectively referred to as the space-time volume. Here, the unit of time corresponds to round of code cycles, which we denote using symbol $\odot$.

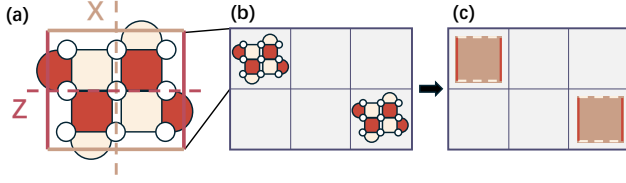


Fig. 2. (a) Example of the distance-3 surface code. (b) Logical qubits are encoded by portions of a device's physical lattice into patches. (c) Abstraction of logical qubits into patches, where the white dashed line represents the X operator and the red line represents the Z operator.

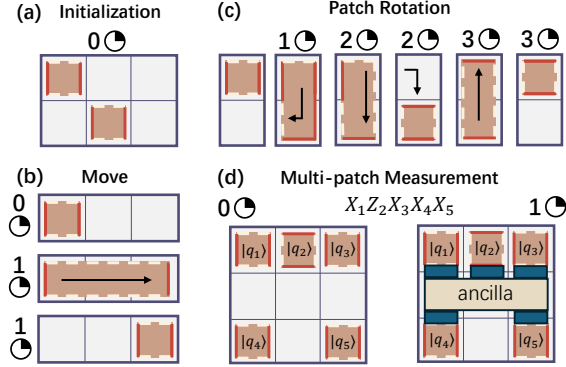


Fig. 3. Patch operations and their related time costs.

Patch Operations. (a) *Initialization.* A single-qubit patch can be initialized in the states $|+\rangle$ or $|0\rangle$ (Fig. 3a), and two-qubit patches can be initialized in $|+\rangle \otimes |+\rangle$ or $|0\rangle \otimes |0\rangle$, all with zero cost ($0\odot$). (b) *Patch Deformation.* A patch can be expanded to cover additional tiles ($1\odot$) or shrunk to occupy fewer tiles ($0\odot$). By combining expansion and shrinkage, a patch can be moved to an adjacent tile (Fig. 3b). (c) *Patch Rotation.* A patch can be rotated by combining corner movements and patch translation operations (Fig. 3c). (d) *Measurement.* The product of Pauli operators can be measured when the relevant edges are adjacent to the *ancilla path* or *routing space*. This operation incurs a cost of $1\odot$ (Fig. 3d). Multi-patch $\pi/4$ and $\pi/8$ measurements can be performed by initializing an ancilla patch A, following the protocol proposed in [34]. A summary of these rules is provided in Fig. 3. For detailed implementation of protocols, we refer to [34].

Representative Data Layouts. (a) *Compact layouts* [34] place qubit patches sequentially within a limited number of rows (e.g., 4 rows and columns in Fig. 4). As more qubits are needed for later applications, additional columns are added to extend the layout, placing new patches (e.g., q_4, q_5) adjacent to existing ones. (b) *Sparse layouts* [25] place both the X - and Z -edges of each data patch adjacent to the routing space, ensuring direct accessibility for logical operations. Each data patch is separated by at least one empty tile from its neighbors. As illustrated in Fig. 4, this results in a sparse layout on a 4×4 board. (c) *Standard layouts* [25] are similar to sparse layouts, but use a different placement, as shown in Fig. 4.

C. Pipeline of Executing Logical Circuits

We introduce the top-down compilation flow following the practices outlined in [5], [25]. The pipeline is shown in Fig. 5.

Step ①: Clifford+ T Decomposition. The program begins with a quantum algorithm written in a high level-language. Since such algorithms are not expressed in the Clifford+ T gate set required for fault-tolerant execution, gate synthesis becomes necessary. This is typically done using the Solovay-Kitaev algorithm [14] or more advanced techniques [42].

Step ②: Transpilation to Pauli-Based Computation. The decomposed Clifford+ T circuits are subsequently transpiled into Pauli product rotations. The Pauli product rotations are defined as $P_\theta = \exp(-iP\theta)$, where P is a multi-qubit Pauli operator. It is equivalent that $S = Z_{\pi/4}$ and $T = Z_{\pi/8}$ and the standard decompositions are given as: $H = Z_{\pi/4}X_{\pi/4}Z_{\pi/4}$ and $CNOT = (Z \otimes X)_{\pi/4}(I \otimes X)_{-\pi/4}(Z \otimes I)_{-\pi/4}$. There are several rules for simplifying circuits based on the commutation relations of Pauli operators. If P and P' commute i.e. $PP' - P'P = 0$, then $P_{\pi/4}$ can be moved past P'_θ . If P and P' anti-commute i.e. $PP' + P'P = 0$, P'_θ turns into $(iPP')_\theta$ when passing $P_{\pi/4}$. Clifford gates can be commuted through the circuit and absorbed into final measurements, as they map Pauli operators to Pauli operators.

Step ③: Surface Code Level Mapping and Scheduling.

After transpilation of Pauli product rotations, one need to perform these instructions by mapping and scheduling following the rules required by lattice surgery. The instructions are initially mapped by assigning logical qubits to different patches of data layout, with the goal of maximizing opportunities for simultaneous multi-patch measurements while minimizing time costs. These instructions are executed sequentially according to the rules outlined in Fig. 3.

D. Motivation

Observation 1 - Existing methods rely on fixed data layout. In most cases, the layout is predefined or designed for executing logical operations. However, these layouts may overlook the scheduling potential based on the specific shapes of patches and often require increasing the overall size to support larger quantum applications. For example, in Fig. 4, consider performing a multi-patch measurement of $Z_0Z_1Z_2Z_3Z_4$ using the compact-style layout, where 'A' denotes the ancilla patch reserved for potential operations. This operation requires 4 time steps due to patch rotation overhead and utilizes 6 ancilla patches to perform the measurements. In addition, the sparse-style layout is invalid in this case because it places an insufficient number of data qubit patches on the board, limiting the available logical resources for computation.

In contrast, irregular data patch placement can potentially improve execution time and reduce space overhead, as indicated by the red cross. Additionally, since the data qubit patches are placed closer together, fewer ancilla patches are needed for routing. For example, Fig. 4 (right) requires only 5 ancilla routing patches. These potential benefits motivate the development of an automated search framework for designing layouts that are difficult to optimize manually.

Example: $Z_0Z_1Z_2Z_3Z_4I_5$ measurement

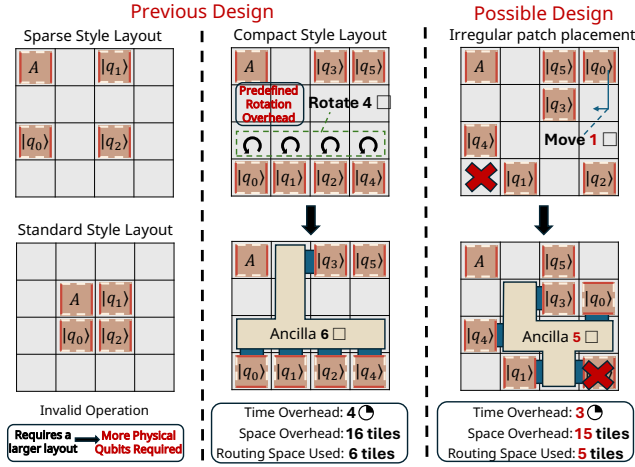


Fig. 4. Example of layout design and loose scheduling (instruction rules based on Fig. 3). Prior sparse or standard style layouts (left) often suffer from inefficient resource usage, while compact style layout (middle) can incur additional scheduling overhead. In this context, irregular designs (right) can achieve higher patch utilization and fewer time steps.

Observation 2 - Current compiler passes' scheduling are static. The term 'static' refers to the current compiler passes that uniformly apply fixed scheduling strategies, regardless of circuit context or layout constraints. For example, in the compact layout, patches are typically rotated to align X and Z operators for multi-patch measurements. However, this approach overlooks cases where such in-place rotations are unnecessary. A concrete example is shown in Fig. 4 (right), where only q_0 is moved downward and the patch is rotated to expose a different edge to the routing space. This avoids the overhead of a full patch rotation and thereby reducing the required time steps. We refer to this more flexible strategy as 'loose' scheduling, as it adapts the placement and orientation of logical qubit patches based on the circuit requirements.

Observation 3 - Potential for Pauli operator cancellation. In a more compact data layout where X and Z operators may not be accessible simultaneously, decomposition of the Y operator becomes necessary. An odd number of Y gates can be decomposed into a single rotation, while an even number must be decomposed into two. The latter enables multiple decomposition schemes, which can be exploited for potential gate cancellation. We present an example in Fig. 6 that illustrates the synthesis of Y operators using X or Z operators. In Fig. 6(a), the previous compiler pass [32], [52] decomposes the rotations $(Y^{\otimes N})_{\pi/8}$ as $[Z_{\pi/4} \otimes (Z^{\otimes N-1})_{\pi/4}](X^{\otimes N})_{-\pi/8}[Z_{-\pi/4} \otimes (Z^{\otimes N-1})_{-\pi/4}]$ for even N , a method that overlooks potential cancellations among Pauli operators. In contrast, Fig. 6(b) shows a case where such cancellations can occur and be utilized in the synthesis process. We instead decomposing $(Y^{\otimes N})_{\pi/8}$ as $[(Z^{\otimes n})_{\pi/4} \otimes (Z^{\otimes N-n})_{\pi/4}](X^{\otimes N})_{-\pi/8}[(Z^{\otimes n})_{-\pi/4} \otimes (Z^{\otimes N-n})_{-\pi/4}]$ where n is odd then $N-n$ is odd. This enables operator elimination and offers potential to reduce time costs.

Observation 4 - Rotations are the primary bottleneck in space-constrained layouts. In space-constrained or irregular data layouts, such as the one shown in Fig. 4, the primary factor limiting execution speed is the need to expose the Z operator to the routing space from the X operator, which requires a rotation operation that takes 3 time steps. This issue is especially evident in irregular layouts, where patches differ in their access to X and Z operators. Some patches expose only a single X or Z operator to the routing space, while others expose both. If a qubit that frequently switches between X and Z operators is mapped to a patch that supports only one of them, the resulting overhead can be significant. This overhead is quantified in Fig. 7, we observe that as the total number of tiles decreases, rotations could become the dominant bottleneck, accounting for more than 50% of the overall time steps. In contrast, using an optimized layout size with moderate tile count, the rotation overhead can be reduced. This motivates us to design improved data layouts that balance tile count and rotation overhead.

III. THE O3LS COMPILER

A. O3LS Module 1: Layout Design

We first propose an algorithm for the automated design of logical qubit data layouts, enabling the search for more squeezed configurations. The core design principle is to preserve the connectivity between the routing space and all data patches, while also maximizing the number of X and Z edges that each data patch exposes to the routing space. This is because the connectivity is intentionally designed to ensure that all data patches are measurable, thereby preventing the failure of logical operations. Moreover, the primary source of time overhead arises from patch rotations. If the layout allows for a greater variety of edge types, this overhead can be reduced, potentially improving overall execution efficiency.

Layout Design Scoring Function. Based on these intuitions, we first design a scoring function S to evaluate the goodness for a given board B . The detailed formulation of the scoring function S is as follows:

$$S(B) = C(B) \times (N_x(B) + N_z(B) - \alpha_e N_e(B)). \quad (1)$$

We use $C(B)$ to indicate whether a routing path exists in the given board or designed layout B that connects to at least one edge of specified data patches. We further require that both the x - and z -edges of the ancilla patch be connected to the routing space, as completing Pauli product rotations necessitates performing Y measurements on the newly initialized ancilla patch [34]. The value of $C(B)$ is 1 if such a routing path exists, and 0 otherwise. We use $N_x(B)$ to denote the number of qubits on board B that are connected to the x -edge by a routing path. Similarly, $N_z(B)$ represents the number of qubits connected to the z -edge. Additionally, we introduce $N_e(B)$ as a penalty term, defined as the number of edges in B that connect data patches to the routing space, and is penalized by the density factor α_e . This term primarily guides the layout design by encouraging the subsequent design process to favor either more compact or sparser layouts.

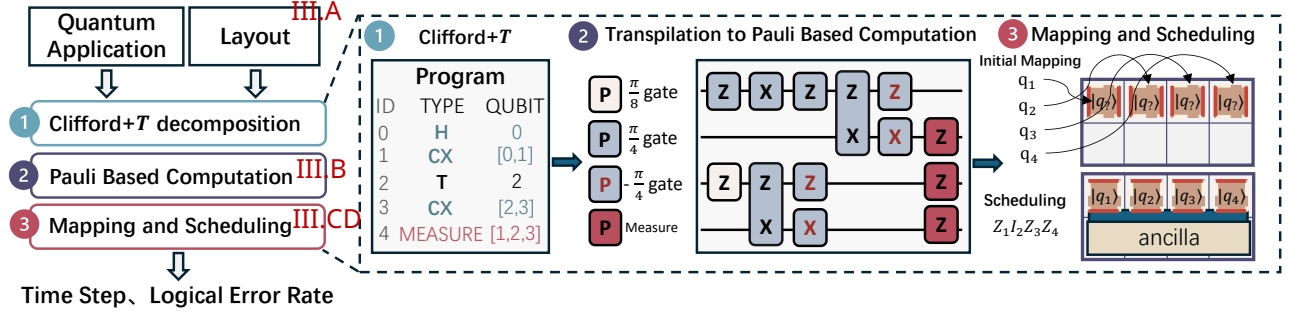


Fig. 5. Pipeline of executing logical circuits. In this work, we introduce an algorithm to optimize layout design, equipped with advanced synthesis techniques for Pauli-based computation, as well as mapping and scheduling strategies to improve both time steps and logical error rate.

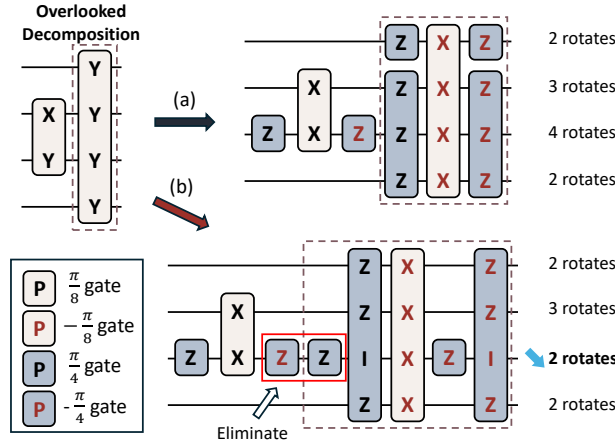


Fig. 6. Operator cancellation opportunities in the process of Pauli-Y decomposition. (a) shows the decomposition method used in prior compiler passes [32], [52], which overlooks the cancellation opportunities shown in (b).

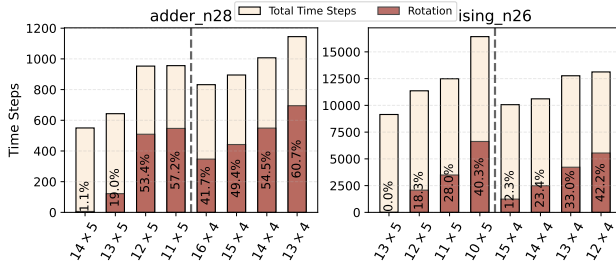


Fig. 7. Quantification of rotation bottleneck (data-layout sizes in x-axis).

Design Process. Based on the scoring function, we propose an iterative layout design method. The board is initialized with an ancilla patch A placed at the corner. Then, at each step, we attempt to add a data patch to the current board B , generating a list of candidate boards $\{B_1^{(1)}, B_2^{(1)}, \dots\}$. Among these candidates, the board with highest-scoring $B_i^{(1)}$ is selected as the updated layout. This process is repeated iteratively until all data patches have been successfully placed. Fig. 8 shows this procedure, where positions indicating the routing path are

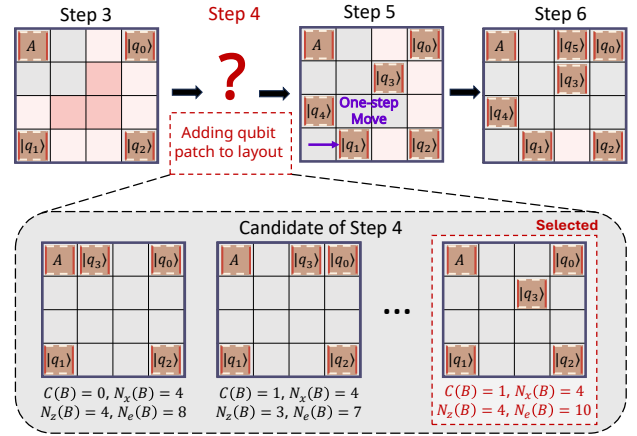


Fig. 8. Layout design process. Darker pink color indicate higher scores. At each step, the highest-scoring position is selected as the new qubit patch.

marked in gray. At each iteration, the location with the highest score is selected for placing the next data patch.

Additionally, after each new qubit patch is placed, we perform a post-processing step to further enhance the overall layout performance. Specifically, this step involves evaluating the potential for relocating existing qubit patches through a one-step move within the current board configuration. If a modified board B' resulting from such a relocation yields a higher score according to the scoring function, then B' is adopted as the updated layout. An example of this process is shown in Step 5 of Fig. 8. After placing q_4 , relocating q_5 increases the number of distinct edges exhibited by q_5 while preserving routing connectivity. This one-step adjustment serves as an effective means of improving the layout's overall performance.

Complexity Analysis. Overall, the computational complexity is $\mathcal{O}(n|B|)$, where n denotes the number of qubits and $|B|$ represents the size of board.

B. O3LS Module 2: Y-Synthesis Algorithm

We then introduce O3LS module 2 to address observation 3: Y-operator decomposition is necessary, and previous compilers

Algorithm 1 Y-synthesis Algorithm

Input: Pauli operator sequence $S = \{P_1, P_2, \dots, P_l\}$
Output: Synthesized Pauli operator sequence S'

```

1: Initialize  $S' = \{\}$ 
2: for  $P_i \in S$  do
3:   Get the Y indices of  $P_i$  as  $Y\_indices_i$ 
4:   if  $size(Y\_indices_i) == 0$  then
5:      $S'.append(P_i)$ 
6:     Continue
7:   else if  $size(Y\_indices_i)$  is odd then
8:     Set  $b_1 = Y\_indices_i$  and  $b_2 = \emptyset$ 
9:   else
10:    Find the bipartition  $b_1, b_2$  of the set  $Y\_indices_i$ .
11:  end if
12:  Build left Z-rotation operator  $L_i^{(1)}, L_i^{(2)}$  and right Z-rotation operator  $R_i^{(1)}, R_i^{(2)}$  according to  $b_1, b_2$ 
13:  Decompose  $P_i$  to get Y-free operator  $P'_i$ 
14:   $S'.append(L_i^{(1)}, L_i^{(2)}, P'_i, R_i^{(1)}, R_i^{(2)})$ 
15: end for
16: Do Pauli operator synthesis on  $S'$ 
17: return  $S'$ 

```

have missed opportunities for Pauli operator cancellation in squeezed data layouts. The key motivation stems from the fact that measuring a Pauli-Y operator requires simultaneous access to both X and Z operators. However, in many layouts with limited data patches, such simultaneous measurements are not feasible, including these generated from O3LS Module 1. This limitation necessitates decomposing Pauli Y operators into equivalent combinations of Pauli X and Z operators.

Y-Synthesis Algorithm. The Y-synthesis algorithm is structured as a two-step process. ① *Y-decompose*. The process referred to as Y-decomposition (e.g. step 3-13 in Algorithm 1) involves transforming Pauli-Y operators into combinations of Pauli-X and Pauli-Z operators, since direct Y-basis measurements are not always supported by the available patch configurations on the surface code lattice. As illustrated in Fig. 9, there may exist multiple valid decomposition schemes for the same operator. Therefore, the Y-decomposition process must select the decomposition that is most suitable for Pauli operator synthesis, particularly in terms of enabling Pauli operator cancellation. ② *Pauli Operator Synthesis*. Pauli operator synthesis (e.g. step 16 in Algorithm 1) is the process of taking a sequence of Pauli operators and merging adjacent ones that share the same Pauli basis to enable cancellation and reduce circuit overhead.

The Y-Synthesis pseudocode is given in Algorithm 1, whose Step 10 is pivotal. Due to the inherent properties of Pauli operators, if an operator contains an even number of Y components, these Y components must be divided into two non-overlapping groups, each containing an odd number of Y components. Subsequently, left Z-rotation and right Z-rotation operators are constructed based on these groups.

During this process, Y-synthesis algorithm evaluates whether any operator derived from a group can be absorbed

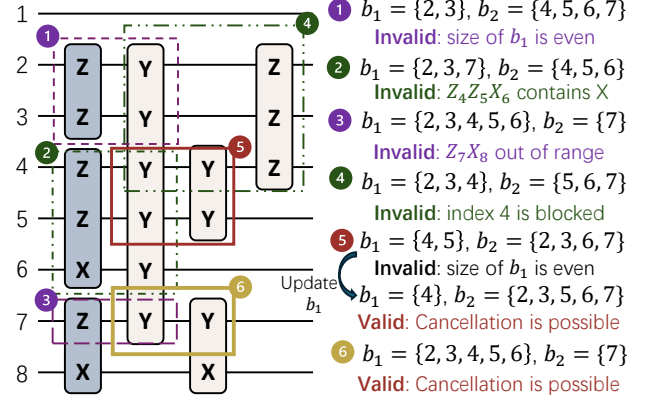


Fig. 9. An example of Y-synthesis rule.

by other operators. Specifically, for each group, we check if at least one operator within that group can be absorbed by another operator. If such an operator exists, the group is designated as a candidate. Among these candidate groups, we then select the one with the highest number of absorbed operators. Fig. 9 is an example of the algorithm, which illustrates the relevant constraints. Since the input Pauli operator sequence is a topologically sorted list, for a specific Pauli operator P_i , its predecessor operators must have already completed the Y-decompose process. Therefore, it is sufficient to ascertain whether the group can be absorbed by its predecessor operators. Additionally, for the successor operators, it is necessary to verify whether there exists a potential Y-decomposition opportunity that could enable the group to be absorbed.

O3LS-IR. To support efficient Y-synthesis in O3LS, we introduce O3LS-IR, an intermediate representation that captures dependencies among Pauli operations to determine execution order and enable parallelism. Pauli operators are represented as nodes in a Pauli Directed Acyclic Graph (PDAG), recorded in our quantum IR, which includes:

- 1) *Rotation angle*: This specifies the rotation angle associated with the Pauli operator. The rotation angle can also indicate whether the node represents a measurement.
- 2) *Pauli words*: These represent the Pauli operators.
- 3) *Predecessor nodes*: These are nodes that precede the current node in the DAG.
- 4) *Successor nodes*: These are nodes that follow the current node in the DAG.

The rules for constructing the PDAG are as follows: Each Pauli operator is represented as a node in the PDAG. A directed edge (P_i, P_j) exists between two nodes P_i and P_j if and only if: (1). There is at least one qubit q on which both P_i and P_j are not the identity operator I . (2). No other Pauli operator P_k between P_i and P_j acts non-trivially on qubit q . A node with in-degree 0 corresponds to an executable Pauli operator. Once executed, the node is removed, and the process repeats until all nodes are processed, ensuring correct dependency resolution.

Complexity analysis. During Y-synthesis, each operator examines at most n predecessor and n successor nodes, resulting in a per-operator computational complexity $\mathcal{O}(n)$. Considering that the Y-decomposition involves at most l Pauli operators, the overall complexity of Algorithm 1 is $\mathcal{O}(nl)$.

C. O3LS Module 3: Loose Scheduling

As highlighted in Observation 2 and illustrated in Fig. 4, most existing approaches rely on pre-defined scheduling patterns or assume that logical patches are sufficiently large to avoid additional patch rotations during multi-patch measurements. This leaves room for further optimization in scheduling. To address this, the O3LS module implements a loose scheduling strategy, where loose refers to the flexibility of the scheduling process. This approach allows for dynamic repositioning and adjustment of patches to better accommodate measurement requirements, ultimately aiming to reduce the total number of time steps. The pseudocode for the loose scheduling algorithm is provided in Algorithm 2.

Algorithm 2 Loose Scheduling Algorithm

Input: Pauli operator sequence $S = \{P_1, P_2, \dots, P_l\}$, qubit number n , Board B

Output: Executable operation sequence S'

```

1: Initialize  $S' = \{\}$ 
2: Build Pauli DAG  $\mathcal{G}$  from  $S$ 
3: while  $\mathcal{G}$  is not empty do
4:   for  $P_i \in \mathcal{G}.\text{frontier}$  and  $P_i$  is executable do
5:     Get bus patch list  $L_{p_i}$  for  $P_i$ 
6:     Execute  $P_i$  on  $L_{p_i}$  and update  $S' \leftarrow S' + P_i$ 
7:   end for
8:   Pop a Pauli operator  $P_j$  from  $\mathcal{G}.\text{frontier}$ 
9:   while  $P_j$  is not executable do
10:    Get all possible patch operations  $O_B$  from  $B$ 
11:    Select  $o_b \in O_B$  with the best reward  $r(o_b, P_j)$ 
12:    Execute  $o_b$  on  $B$  and update  $S' \leftarrow S' + o_b$ 
13:   end while
14: end while
15: return  $S'$ 

```

Determining bus patch. In Step 5, the objective is to identify the minimal amount of routing space required to execute P_i , as reducing the overall routing path length not only increases opportunities for parallel execution of other operations but also has the potential to lower the overall logical error rate. To achieve this, we apply Dijkstra's algorithm to sequentially determine the shortest paths between the required patches. Previously identified paths are treated as nodes with zero cost. This process results in a bus patch list with a minimal number of patches used and the complexity of each executed Pauli operator P_i is $\mathcal{O}(|B|^2)$, where $|B|$ denotes the size of board.

Resolving unexecutable Pauli operator. In Step 11, the reward function is decomposed into three components. First, it is defined as the number of data patches in B_o that enable the execution of P_j ; that is the number of data patches for

which a valid path exists to support the application of P_j , where B_o denotes the state of the board B after applying the patch operation o . Second, maintaining connectivity among all data patches is essential for enabling subsequent lattice surgery operations, so any patch operation that breaks this connectivity receives a reward of zero. Third, when multiple patch operations yield the same reward, preference is given to those incur lower time overhead.

Complexity Analysis. Since the number of candidate patch operations for each data patch is constant, the total number of candidates is $\mathcal{O}(n)$, where n denotes the number of data patches. Also, the complexity of evaluating the reward function is also $\mathcal{O}(n)$. Furthermore, the reward function is designed to guarantee that each patch operation increases the number of patches satisfying the execution requirements by at least one. Consequently, the overall complexity of a complete scheduling process is $\mathcal{O}(n^2)$.

D. O3LS Module 4: Initial Mapping

Edge-aware Initial Mapping. Among all patch operations, qubit patch rotations are a major source of time overhead, particularly in squeeze layouts where such operations occur more frequently. To mitigate this overhead, we propose an edge-aware (EA) initial mapping strategy. The core idea is to analyze the PDAG to estimate the rotation demand for each qubit. Qubits with higher expected rotation frequencies are preferentially mapped to patches that are adjacent to both the X and Z-edges of the ancilla patch, thereby minimizing the need for costly patch rotations.

The effectiveness of the EA mapping method is most evident in layout boards that fall between compact and sparse configurations. For instance, in highly compact layouts, changes in qubit positions introduced by the mapping are less likely to expose additional edges, thereby limiting the potential benefits of the approach. In these cases, the performance of the EA strategy becomes highly dependent on the specific structure and gate distribution of the input circuit. Moreover, in sparse qubit patch layouts, the edge-aware component contributes less to performance improvement. This is because, in sparse configurations, each patch can typically expose both X and Z operators simultaneously, reducing the advantage gained from edge-aware placement.

Complexity Analysis. The rotation demand required for EA mapping can be efficiently extracted during the construction of PDAG. The process of counting the number of edges exposed by data patches has a computational complexity of $\mathcal{O}(n)$, where n denotes the number of data patches. The final mapping is achieved through two applications of quicksort, resulting in an overall complexity of $\mathcal{O}(n \log n)$.

IV. EVALUATION

A. Experiment Setup

a) Metrics. (1). *Logical error rate.* We simulate the logical error rate (LER) by parsing lattice surgery instructions and analyzing them at the time-slice level. We then compute the

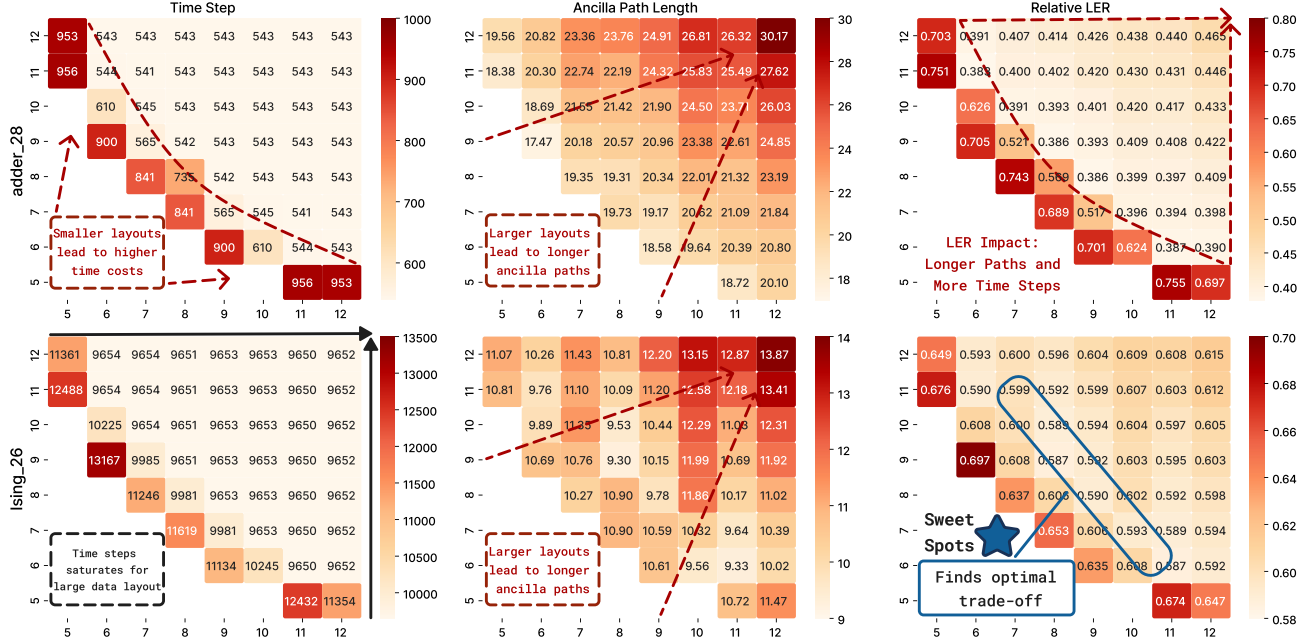


Fig. 10. Performance of executing applications on various data layout sizes using O3LS, including their corresponding time steps, ancilla patch lengths, and logical error rates for adder_28 and ising_26. The X and Y axes represent patch boards of size $N \times M$, where N and M range from 5 to 12.

layer-wise LER for each time slice and linearly accumulate them over the entire execution:

$$p_{\text{total}} \approx \sum_{t=1}^T p_{\text{layer}}^{(t)} \approx \sum_{t=1}^T (1 - (1 - P_{\text{PPM}}^{(t)})(1 - P_{\text{PR}}^{(t)})(1 - P_{\text{idle}}^{(t)})),$$

which follows the same method used in SPARO [28] under the assumption of rare failures and independent error events. Each $p_{\text{layer}}^{(t)}$ is estimated from the simulated LER in Pauli product measurement (PPM), patch rotation (PR), and idling memory errors during that layer. The PPM error rate is mainly determined by the routing space and the code distance. The rotation step is decomposed into three slices: patch deformation, corner movement, and patch movement [34], and they are simulated separately. (2). *Time cost*. Following prior work [25], [51], our evaluation also focuses on the time cost (⊙). We also record the size of data layout, which serves as an indicator of the space-time volume [47].

b) Baseline Algorithms. We evaluate O3LS against the SPC algorithm [34], as implemented in [32], [52], and the recent locality-aware method LAPBC [25], which enhances circuit parallelism and outperforms prior compilers [4], [39]. We also compare against SPARO [28], another automated data-layout design method that aims to expand data layouts.

c) Benchmarks. We benchmark using a representative set of FT quantum algorithms, following prior FTQC compiler studies [28], [38], [39], [48], [51]. These include circuits for Hamiltonian simulation, Quantum Fourier Transform, key components of Shor’s algorithm (e.g., adders and multipliers), and SWAP tests for quantum machine learning, *many of which serve as building blocks for larger algorithms*. We source the

QASM files from MQT Bench [41] and FTCircuitBench [21]. Some FTCircuitBench circuits were originally taken from QASMBench [33]. Unless otherwise specified in the circuit name, we assume a one-dimensional Hamiltonian. We evaluate O3LS across different layouts, including the compact design from [34] and standard layouts from [25].

d) Experimental Setting. The benchmarks are decomposed into Clifford+ T circuits using GridSynth [13], based on [42] with a synthesis error tolerance of 10^{-5} . STIM simulations [18] are conducted to characterize atomic lattice surgery operations using a $d = 9$ surface code under a circuit-level depolarizing noise model with a physical error rate of $p = 10^{-3}$. Each atomic operation is independently compiled into a STIM circuit and simulated using Monte Carlo sampling with no less than 10^6 trials. Decoding is performed by PyMatching 2 [24]. For all experiments, we use a magic-state factory based on the 15-to-1 distillation protocol [8]. The factory is placed outside the designed layout, while ensuring at least one routing path connects it to the data region. The $\pi/4$ and $\pi/8$ Pauli-product measurements are implemented via standard gate teleportation protocol, following the implementation in [34] (Fig. 7 and 11(b)).

e) Simulation Device. All simulations were performed on a device with an Intel Core i9-14900K 32-core processor and 188 GB of RAM using Python 3.10.

B. Analysis of Data Layout Designs

a) Performance on Data Layout Designs. We begin by analyzing the performance across different data layout sizes and their corresponding estimated logical error rates. In this

scenario, we consider patch boards of size $N \times M$, where N and M range from 5 to 12, and evaluate the performance of adder_28 and Ising_26 circuits across various patch board configurations. The experimental results are presented in Fig. 10, where the heatmaps illustrate the time steps, ancilla path lengths, and their associated RLER.

Our evaluation reveals that when the patch board size is too small, the primary performance bottleneck arises from the overhead associated with operation scheduling, such as patch rotations. As the board size increases, the number of time steps required for execution decreases and eventually converges. However, despite similar time step at larger board sizes, the ancilla patch length increases monotonically due to the availability of more routing space. This leads to an observation that both higher time step and longer ancilla path lengths contribute significantly to increased logical error rates. *These results underscore a fundamental trade-off between time costs and ancilla patch distance, indicating that carefully designed, smaller data layouts can contribute to lower logical error rates.*

On the other hand, the applications that are required to run on these fixed data layouts necessitate the use of 10×10 and 9×15 patch boards in the standard and sparse layouts. Compared to these scenarios, O3LS effectively reduces the required space overhead, achieving a board size reduction of up to 28.0% and 46.7%, while preserving the number of time steps. Furthermore, it achieves a reduction in logical error rates of up to 16.9% compared to larger data layouts (e.g., 12×12), due to the decreased length of ancilla patches. While larger data layouts can reduce the number of time steps, they often require longer routing paths. In contrast, smaller data layouts are more space-efficient but tend to suffer from higher time costs. Both scenarios can contribute to increased logical error rates. To balance this trade-off, *O3LS generates more compact layout designs that minimize ancilla patch length while maintaining time costs comparable to those of sparse layouts, thereby achieving a sweet spot for reducing the overall logical error rate.*

b) Sensitivity Analysis on Density Factor. We also perform a sensitivity analysis of the density factor α_e in the layout design, as introduced in Section III-A, with results presented in Fig. 11. The experiments indicate that values of α_e between 0.1 and 0.3 yield the best performance across most applications. Additionally, performance remains relatively stable for $0 < \alpha_e < 0.5$, but degrades noticeably when $\alpha_e = 0$ or $\alpha_e = 0.5$. This behavior can be attributed to the trade-off governed by α_e : smaller values prioritize placing patches with multiple edges to ancilla patches, promoting sparse layouts, while larger values emphasize compactness. Both extremes can lead to suboptimal layouts—either overly sparse and causing later qubit patches to obscure certain operators or overly compact and underutilizing available patches. Thus, an appropriate balance of α_e is essential to optimize the trade-off between compact and sparse placement in patch utilization. We recommend using $\alpha_e \in [0.1, 0.3]$ to achieve this balance.

c) Comparison with SPARO. Moreover, other layout-

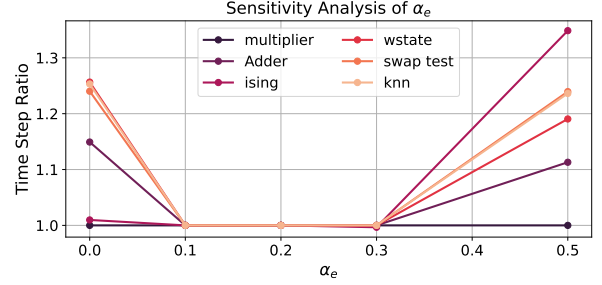


Fig. 11. Sensitivity analysis of density factor on automatic layout design. The relative time step is defined as the ratio between the evaluated time step and the baseline time step corresponding to density factor $\alpha_e = 0.1$.

design methods such as SPARO [28] explore strategies to improve data-layout utilization. In Fig. 12, we compare O3LS with SPARO’s layout-design approach. ‘O3LS-1’ uses an O3LS-generated data layout with SPARO’s scheduling method and ‘O3LS’ uses the full O3LS stack for both layout generation and scheduling.

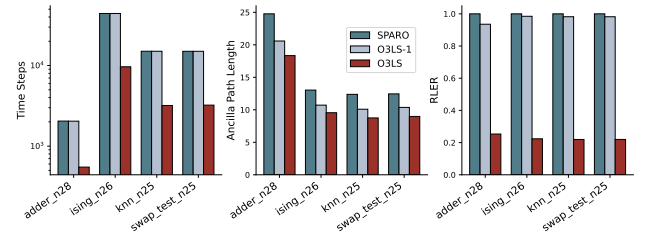


Fig. 12. Layout design comparison with SPARO [28].

O3LS-generated data layouts achieve better performance than SPARO, improving LER by 3.05% on average when paired with SPARO’s scheduling method. This gain comes primarily from smaller data layouts, which reduce average ancilla-routing space by 17.35%. In addition, O3LS’s scheduler is particularly effective on the smaller data layouts produced by O3LS Module 1. It reduces time steps by 78.24% and average routing space by 27.17% on average, which together yield a further 77.1% reduction in LER. The routing-space savings in O3LS largely stem from its objective. O3LS searches for a sweet spot that minimizes space overhead in the data layout, whereas SPARO tends to allocate more data-layout resources. O3LS also includes more advanced synthesis and a looser scheduling strategy, which further reduces time steps. Overall, O3LS finds smaller layouts that shorten ancilla paths while reducing time steps, resulting in lower LER.

C. Compilation Technique Comparison

a) Comparison with SPC. We begin by comparing our compiler pass with SPC across two data layout configurations: compact and standard. As shown in Fig. 13, our compiler pass O3LS achieves an average reduction of 36.07% in time steps compared to SPC under the compact layout, and an

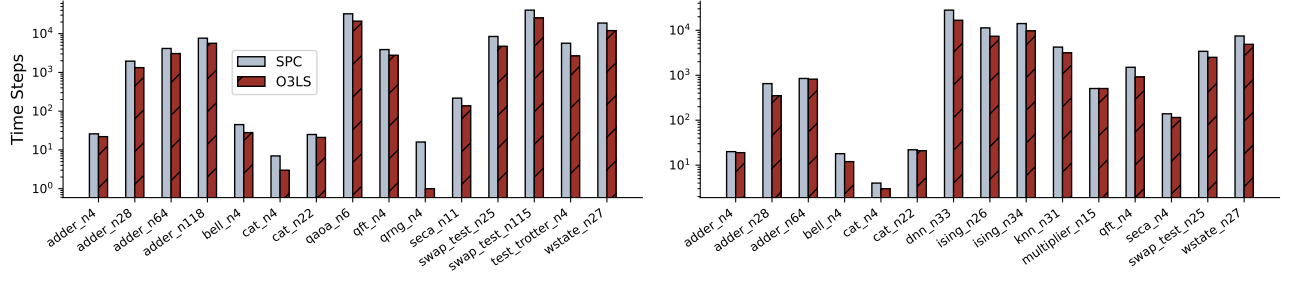


Fig. 13. Compilation technique comparison with SPC. (Left) Results with the compact layout. (Right) Results with the standard layout.

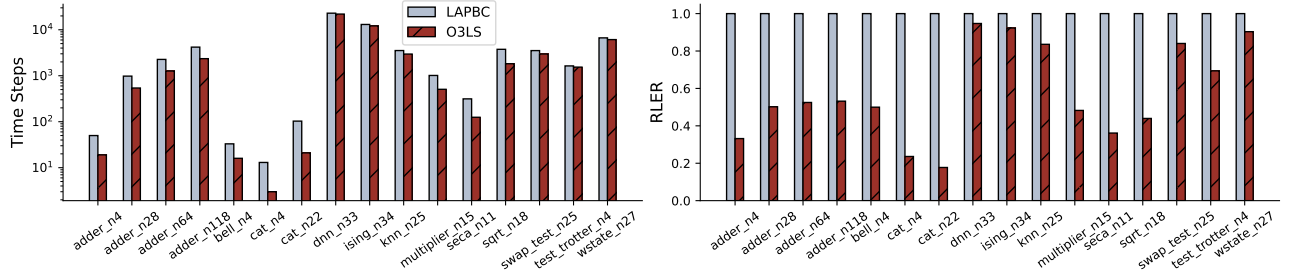


Fig. 14. (Left) Comparison of time steps between the prior compiler pass LAPBC, which emphasizes parallelism, and our proposed O3LS. (Right) Logical error rate analysis indicates that O3LS achieves superior performance in most scenarios, highlighting the effectiveness of the proposed scheduling and Y-synthesis algorithms.

average reduction of 24.76% under the standard layout. These improvements achieved by O3LS are due to the fact that SPC does not incorporate any optimization techniques, such as scheduling, routing, or synthesis.

b) Comparison with LAPBC. Recent efforts have focused on improving circuit parallelism in lattice surgery [4], [5], [20], [25], with LAPBC [25] as the latest advancement. In Fig. 14 (left), we analyze the time steps required by both compilers. The results show that O3LS achieves an average time-step reduction of 35.10% compared to LAPBC, with a maximum reduction of up to 80.6%. This results in an average LER reduction of 38.8%, with a maximum reduction of up to 82.3%, as shown in Fig. 14 (right). This highlights the effectiveness of O3LS in optimizing execution schedules with loose scheduling algorithm.

c) Incorporating parallelism into O3LS. While the highly parallel nature of LAPBC provides advantages for sparse data layouts, O3LS can also be integrated into LAPBC to further reduce time costs. In particular, we focus on several high-parallelism benchmarks where LAPBC is expected to perform well. By incorporating additional modules such as loose scheduling and advanced initial mapping algorithms, we further improve performance and demonstrate the effectiveness of the proposed methods. A detailed breakdown of the results is shown in Fig. 15, where our integrated approach achieves an average improvement of 9.31%.

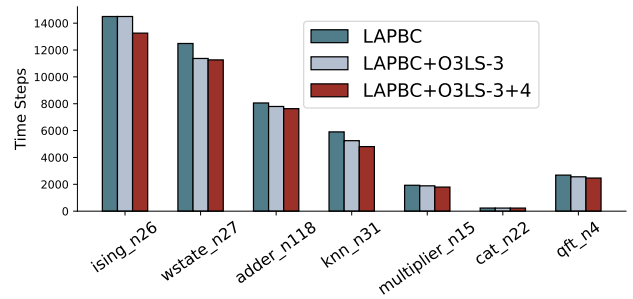


Fig. 15. Time step reduction by integrating O3LS with high-parallelism execution strategies.

D. Initial Mapping Comparison

Furthermore, we analyze the initial mapping methods proposed in O3LS and compare them with the previous greedy mapping approach from [28]. We use the data layouts generated by O3LS to evaluate these initial mappings. As shown in Fig. 16, the edge-aware mapping outperforms the previous approach, achieving a time step reduction of 15.0% and a logical error rate reduction of 8.4%. These savings primarily result from the edge-aware mapping's tendency to place qubits with higher rotation demands in patches where both X and Z operators are adjacent to an ancilla patch, thereby reducing the need for costly rotations. This strategy is especially effective for the squeezed layouts produced by O3LS.

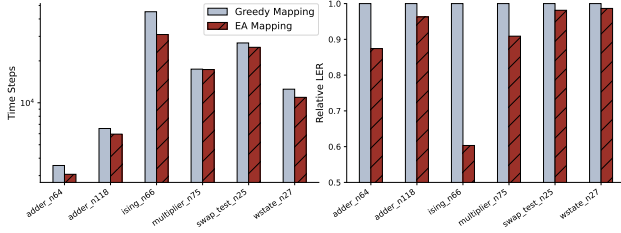


Fig. 16. Initial mapping comparison.

E. Overall Performance Comparison

a) Overall Performance. After comparing individual components, we directly compare the overall performance of O3LS with compiler passes including SPC and LAPBC in terms of LER. The exact LER results are presented in Fig. 18. Compared to SPC, O3LS suppresses the logical error rate by 43.11% and 44.98% on compact and standard layouts, respectively (corresponding to a reduction by roughly half). Notably, in certain cases, O3LS achieves a maximum LER reduction of 93.95% compared to LAPBC, approaching an order of magnitude error suppression.

Since the simulated surface code distances and the level of improvement vary across applications, we also record the results and normalize them into relative LER values by setting the highest LER for each application as the baseline to clearly show the improvements. Overall, O3LS outperforms SPC, reducing relative LER by 35.9% and 21.8% on compact and standard layouts, respectively. Similarly, compared to LAPBC, O3LS achieves relative LER reductions of 50.9% and 31.1%.

Overall, O3LS outperforms previous compiler passes primarily due to all module design choices. First, it identifies the trade-off between scheduling overhead and ancilla path length during layout design, leading to more efficient data layouts. It also applies circuit synthesis techniques to reduce the total number of operations, thereby lowering time overhead associated with rotating data patches in the O3LS-generated layouts. Second, rather than relying on fixed scheduling schemes, O3LS introduces a flexible scheduling strategy that minimize scheduling costs. Third, it incorporates effective initial mapping techniques that improve both the routing efficiency. Together, these strategies not only reduce time overhead but also minimize the need for long ancilla paths, resulting in suppression of LER.

b) Sensitivity Analysis on Code Distance. Fig. 19 shows a sensitivity analysis over surface code distance $d \in [3, 5, 7, 9]$. The results show that O3LS consistently outperforms all previous compilers across all tested code distances. For the Ising_n26 and swap_test_n25 benchmark, O3LS consistently achieves over 19.96% and 13.42% improvement in RLER compared to SPC and LAPBC, respectively. Crucially, this relative improvement remains stable as the code distance increases. This is because, under the layer-wise accumulation model that treats logical failures as independent rare events [28], O3LS optimizes only architectural factors, while

the distance-dependent exponential suppression from decoding applies equally to O3LS and the baseline. Consequently, our gains are not tied to any particular code distance.

c) Analysis on Resource Estimation. After demonstrating robustness across code distances and noise models in improving LER for all benchmarks, Fig. 20 (upper) reports resource estimates for overall time (number of syndrome-measurement cycles) and space (physical qubit count) savings achieved by O3LS relative to SPC. Overall, O3LS reduces space and time simultaneously, delivering an average 23.63% improvement compared with the prior compiler and thereby suppressing LER (Fig. 20 (lower right)) through automated layout design and scheduling.

Fig. 20 (lower left) further quantifies the space savings using the surface code with $d = 9$ as an example. Because each tile corresponds to one surface-code logical qubit, reducing the tile count directly reduces the number of physical qubits. In our benchmarks, O3LS achieves up to a 44% space reduction, which corresponds to saving roughly 7000 physical qubits. This benefit becomes even more pronounced at larger surface-code distances. This highlights the usefulness of O3LS, which reduces not only time steps but also the physical-qubit requirements for executing fault-tolerant algorithms on hardware.

F. Ablation Study of Compilation Techniques

Fig. 21 presents a comparative analysis of the individual modules in O3LS, using the data layout generated by the ‘O3LS-1’ configuration as the test case. ‘O3LS-2’ refers to the compiler pass that utilizes only the Module 2 Y-synthesis algorithm described in Sec. III-B. Compared with prior compiler passes, ‘O3LS-2’ improves time steps by 18.33% and LER by 18.30%, highlighting the potential of operator cancellation for optimizing circuit execution. With the integration of loose scheduling, ‘O3LS-2+3’ achieves an average improvement of 37.74% in time steps and 34.34% in LER, demonstrating the added benefit of loose scheduling. Finally, incorporating initial mapping technique in ‘O3LS-2+3+4’ further improves performance, yielding an average improvement of 38.62% in time steps and 35.17% in LER. The results highlight that combining Y-synthesis, loose scheduling, and edge-aware mapping reduces execution time and improves lattice surgery compilation efficiency.

G. Compilation Time Analysis

We compare the compilation time across different compilers in Fig. 22 (left). O3LS achieves faster compilation times than SPC and delivers comparable performance to LAPBC. In some cases, O3LS is slightly slower than LAPBC, which benefits from maximizing parallelism and avoiding the overhead of absorbing Pauli operators into the final measurement. Although O3LS performs explicit Pauli operator transformations, it leverages the O3LS-IR to accelerate this process more effectively than SPC, resulting in compilation times that remain competitive with LAPBC. In Fig.22 (right), we also demonstrate the scalability of O3LS with respect to the

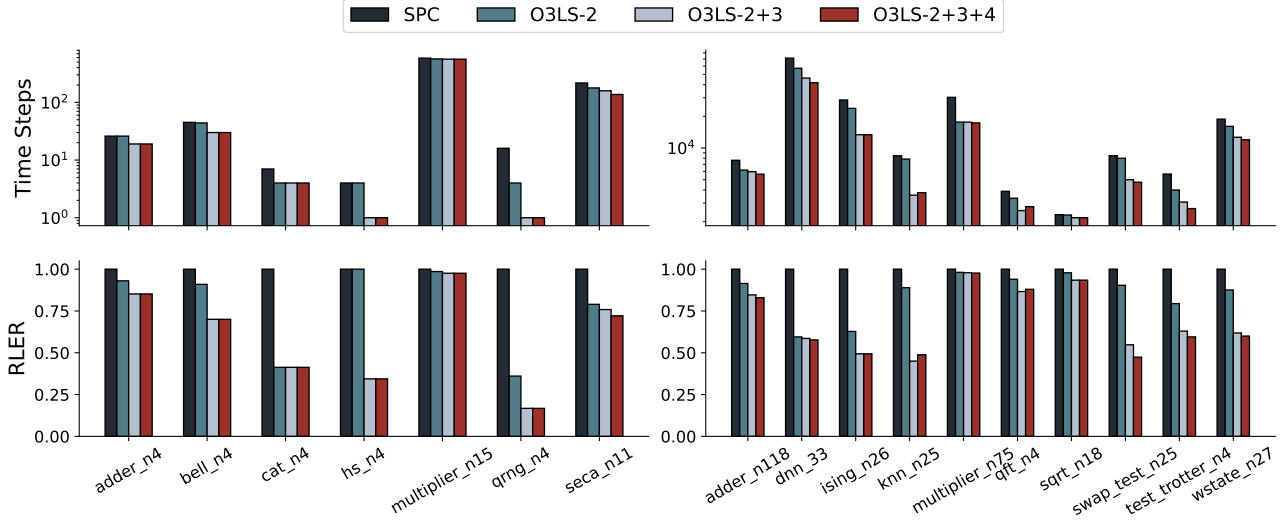


Fig. 21. Ablation study of compilation techniques. O3LS-2 refers to the use of Y-synthesis algorithm without additional scheduling methods. O3LS-2+3 incorporates loose scheduling in addition to Y-synthesis, while O3LS-2+3+4 integrates both initial mapping and loose scheduling alongside the Y-synthesis.

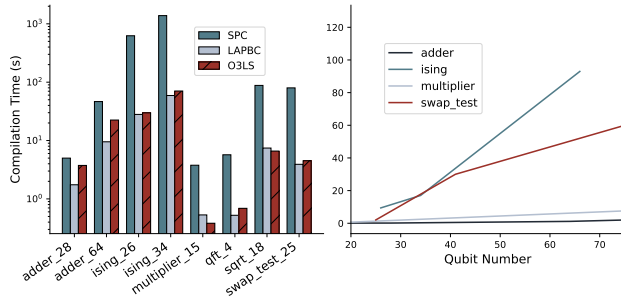


Fig. 22. Compilation time analysis. (Left) Comparison with previous compilers. (Right) Scalability with respect to the number of logical qubits.

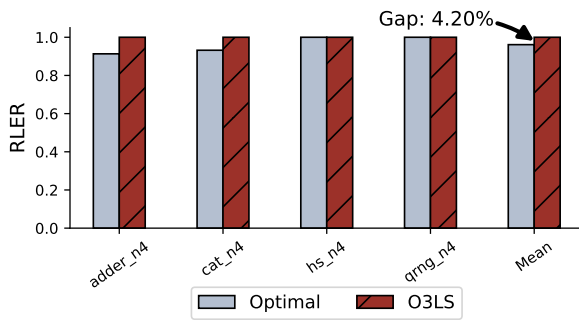


Fig. 23. Optimality analysis.

enlarges the underlying data layout based on the analyzed bottleneck. LaSynth [47] proposes a SAT-based solver that can optimally handle a limited number of qubits and operations. However, its scalability is limited. Our aim is to develop an automated and scalable compiler for finding squeezed layouts.

Heterogeneous QEC designs. [45] introduces a toolbox for heterogeneous quantum architectures on superconducting devices, while [6], [46], [54] propose hybrid approaches that combine surface codes with qLDPC codes, leveraging their complementary strengths by assigning different codes to memory and computation regions. In all cases, surface-code architectures remain central, and our work can potentially offer improved pipelines to enhance their performance.

d) Mapping QEC codes into hardware. [53] presents a synthesis framework for surface codes on superconducting devices, while [57] extends this to stabilizer code mapping. [31], [56] explore surface code mapping on trapped-ion devices. These studies primarily focus on the lower layer of mapping QEC codes to physical hardware. Our work is orthogonal to these efforts, and integrating both layers has the potential to further reduce logical error rates.

VI. CONCLUSION

We present O3LS, a compiler that suppresses logical error rates by optimizing both space and time overhead for lattice surgery operations. It produces data layouts that minimize space overhead while maintaining time costs comparable to those of sparser layouts, thereby supporting the goal of reducing logical error rates. O3LS achieves this through loose scheduling, Y operator synthesis, and initial mapping tailored to the proposed layout architecture. The numerical results demonstrate that O3LS could outperform prior works in terms of error rates, time costs, and qubit resource overhead.

ACKNOWLEDGMENT

We would like to thank the anonymous reviewers for their helpful feedback and suggestions.

REFERENCES

- [1] R. Acharya, D. A. Abanin, L. Aghababaie-Beni, I. Aleiner, T. I. Andersen, M. Ansmann, F. Arute, K. Arya, A. Asfaw, N. Astrakhantsev *et al.*, “Quantum error correction below the surface code threshold,” *Nature*, 2024.
- [2] G. Q. AI *et al.*, “Quantum error correction below the surface code threshold,” *Nature*, vol. 638, no. 8052, p. 920, 2024.
- [3] F. Arute, K. Arya, R. Babbush, D. Bacon, J. C. Bardin, R. Barends, R. Biswas, S. Boixo, F. G. Brandao, D. A. Buell *et al.*, “Quantum supremacy using a programmable superconducting processor,” *Nature*, vol. 574, no. 7779, pp. 505–510, 2019.
- [4] M. Beverland, V. Kliuchnikov, and E. Schoute, “Surface code compilation via edge-disjoint paths,” *PRX Quantum*, vol. 3, no. 2, p. 020342, 2022.
- [5] M. E. Beverland, P. Murali, M. Troyer, K. M. Svore, T. Hoefler, V. Kliuchnikov, G. H. Low, M. Soeken, A. Sundaram, and A. Vassillo, “Assessing requirements to scale to practical quantum advantage,” *arXiv preprint arXiv:2211.07629*, 2022.
- [6] S. Bravyi, A. W. Cross, J. M. Gambetta, D. Maslov, P. Rall, and T. J. Yoder, “High-threshold and low-overhead fault-tolerant quantum memory,” *Nature*, vol. 627, no. 8005, pp. 778–782, 2024.
- [7] S. Bravyi and J. Haah, “Magic-state distillation with low overhead,” *Physical Review A*, vol. 86, no. 5, Nov. 2012. [Online]. Available: <http://dx.doi.org/10.1103/PhysRevA.86.052329>
- [8] S. Bravyi and A. Kitaev, “Universal quantum computation with ideal clifford gates and noisy ancillas,” *Physical Review A*, vol. 71, no. 2, Feb. 2005. [Online]. Available: <http://dx.doi.org/10.1103/PhysRevA.71.022316>
- [9] L. Caune, L. Skoric, N. S. Blunt, A. Ruban, J. McDaniel, J. A. Valery, A. D. Patterson, A. V. Gramolin, J. Majaniemi, K. M. Barnes *et al.*, “Demonstrating real-time and low-latency quantum error correction with superconducting qubits,” *arXiv preprint arXiv:2410.05202*, 2024.
- [10] M. Cerezo, A. Arrasmith, R. Babbush, S. C. Benjamin, S. Endo, K. Fujii, J. R. McClean, K. Mitarai, X. Yuan, L. Cincio *et al.*, “Variational quantum algorithms,” *Nature Reviews Physics*, vol. 3, no. 9, pp. 625–644, 2021.
- [11] C. Chamberland and E. T. Campbell, “Universal quantum computing with twist-free and temporally encoded lattice surgery,” *PRX Quantum*, vol. 3, no. 1, p. 010331, 2022.
- [12] A. Chatterjee, A. Ghosh, and S. Ghosh, “The q-spellbook: Crafting surface code layouts and magic state protocols for large-scale quantum computing,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.11253>
- [13] Q. Community, “qiskit-gridsynth-plugin: A plugin for integrating gridsynth angle decomposition into qiskit,” <https://pypi.org/project/qiskit-gridsynth-plugin/>, 2024, version 0.0.8, released August 29, 2024.
- [14] C. M. Dawson and M. A. Nielsen, “The solovay-kitaev algorithm,” 2005. [Online]. Available: <https://arxiv.org/abs/quant-ph/0505030>
- [15] S. Ebadi, A. Keesling, M. Cain, T. T. Wang, H. Levine, D. Bluvstein, G. Semeghini, A. Omran, J.-G. Liu, R. Samajdar *et al.*, “Quantum optimization of maximum independent set using rydberg atom arrays,” *Science*, vol. 376, no. 6598, pp. 1209–1215, 2022.
- [16] A. Eickbusch, M. McEwen, V. Sivak, A. Bourassa, J. Atalaya, J. Claes, D. Kafri, C. Gidney, C. W. Warren, J. Gross *et al.*, “Demonstrating dynamic surface codes,” *arXiv preprint arXiv:2412.14360*, 2024.
- [17] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, “Surface codes: Towards practical large-scale quantum computation,” *Physical Review A—Atomic, Molecular, and Optical Physics*, vol. 86, no. 3, p. 032324, 2012.
- [18] C. Gidney, “Stim: a fast stabilizer circuit simulator,” *Quantum*, vol. 5, p. 497, 2021.
- [19] D. Gottesman, *Stabilizer codes and quantum error correction*. California Institute of Technology, 1997.
- [20] K. Hamada, Y. Suzuki, and Y. Tokunaga, “Efficient and high-performance routing of lattice-surgery paths on three-dimensional lattice,” 2024. [Online]. Available: <https://arxiv.org/abs/2401.15829>
- [21] A. Harkness, S. Kan, C. Liu, M. Wang, J. M. Martyn, S. Xu, D. Chamaki, E. Decker, Y. Mao, L. F. Zuluaga, T. Terlaky, A. Li, and S. Stein, “Ftcircuitbench: A benchmark suite for fault-tolerant quantum compilation and architecture,” 2026. [Online]. Available: <https://arxiv.org/abs/2601.03185>
- [22] A. W. Harrow, A. Hassidim, and S. Lloyd, “Quantum algorithm for linear systems of equations,” *Physical review letters*, vol. 103, no. 15, p. 150502, 2009.
- [23] D. Herr, F. Nori, and S. J. Devitt, “Optimization of lattice surgery is np-hard,” *Npj quantum information*, vol. 3, no. 1, p. 35, 2017.
- [24] O. Higgott and C. Gidney, “Sparse blossom: correcting a million errors per core second with minimum-weight matching,” *Quantum*, vol. 9, p. 1600, 2025.
- [25] Y. Hirano and K. Fujii, “Locality-aware pauli-based computation for local magic state preparation,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.12091>
- [26] D. Horsman, A. G. Fowler, S. Devitt, and R. Van Meter, “Surface code quantum computing by lattice surgery,” *New Journal of Physics*, vol. 14, no. 12, p. 123011, 2012.
- [27] C. Jones, “Multilevel distillation of magic states for quantum computing,” *Physical Review A*, vol. 87, no. 4, Apr. 2013. [Online]. Available: <http://dx.doi.org/10.1103/PhysRevA.87.042305>
- [28] S. Kan, Z. Du, C. Liu, M. Wang, Y. Ding, A. Li, Y. Mao, and S. Stein, “Sparo: Surface-code pauli-based architectural resource optimization for fault-tolerant quantum computing,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.21854>
- [29] S. Krinner, N. Lacroix, A. Remm, A. Di Paolo, E. Genois, C. Leroux, C. Hellings, S. Lazar, F. Swiadek, J. Herrmann *et al.*, “Realizing repeated quantum error correction in a distance-three surface code,” *Nature*, vol. 605, no. 7911, pp. 669–674, 2022.
- [30] L. Lao, B. van Wee, I. Ashraf, J. Van Someren, N. Khammassi, K. Bertels, and C. G. Almudever, “Mapping of lattice surgery-based quantum circuits on surface code architectures,” *Quantum Science and Technology*, vol. 4, no. 1, p. 015005, 2018.
- [31] T. LeBlond, R. S. Bennink, J. G. Lietz, and C. M. Seck, “Tisc: A surface code compiler and resource estimator for trapped-ion processors,” in *Proceedings of the SC’23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis*, 2023, pp. 1426–1435.
- [32] T. LeBlond, C. Dean, G. Watkins, and R. Bennink, “Realistic cost to execute practical quantum circuits using direct clifford+ t lattice surgery compilation,” *ACM Transactions on Quantum Computing*, 2023.
- [33] A. Li, S. Stein, S. Krishnamoorthy, and J. Ang, “Qasmbench: A low-level quantum benchmark suite for nisy evaluation and simulation,” *ACM Transactions on Quantum Computing*, vol. 4, no. 2, pp. 1–26, 2023.
- [34] D. Litinski, “A game of surface codes: Large-scale quantum computing with lattice surgery,” *Quantum*, vol. 3, p. 128, Mar. 2019. [Online]. Available: <http://dx.doi.org/10.22331/q-2019-03-05-128>
- [35] —, “Magic state distillation: Not as costly as you think,” *Quantum*, vol. 3, p. 205, Dec. 2019. [Online]. Available: <http://dx.doi.org/10.22331/q-2019-12-02-205>
- [36] M. Liu, R. Shaydulin, P. Niroula, M. DeCross, S.-H. Hung, W. Y. Kon, E. Cervero-Martín, K. Chakraborty, O. Amer, S. Aaronson *et al.*, “Certified randomness using a trapped-ion quantum processor,” *Nature*, pp. 1–6, 2025.
- [37] J. F. Marques, B. Varbanov, M. Moreira, H. Ali, N. Muthusubramanian, C. Zachariadis, F. Battistel, M. Beekman, N. Haider, W. Vlothuizen *et al.*, “Logical-qubit operations in an error-detecting surface code,” *Nature Physics*, vol. 18, no. 1, pp. 80–86, 2022.
- [38] S. Maurya, A. Molavi, A. Albarghouthi, and S. Tannu, “Managing classical processing requirements for quantum error correction,” *arXiv preprint arXiv:2406.17995*, 2024.
- [39] A. Molavi, A. Xu, S. Tannu, and A. Albarghouthi, “Dependency-aware compilation for surface code quantum architectures,” *Proceedings of the ACM on Programming Languages*, vol. 9, no. OOPSLA1, pp. 57–84, 2025.
- [40] J. Preskill, “Quantum computing in the nisy era and beyond,” *Quantum*, vol. 2, p. 79, 2018.
- [41] N. Quetschlich, L. Burgholzer, and R. Wille, “Mqt bench: Benchmarking software and design automation tools for quantum computing,” *Quantum*, vol. 7, p. 1062, Jul. 2023. [Online]. Available: <http://dx.doi.org/10.22331/q-2023-07-20-1062>
- [42] N. J. Ross and P. Selinger, “Optimal ancilla-free clifford+t approximation of z-rotations,” 2016. [Online]. Available: <https://arxiv.org/abs/1403.2975>
- [43] P. W. Shor, “Scheme for reducing decoherence in quantum computer memory,” *Physical review A*, vol. 52, no. 4, p. R2493, 1995.

- [44] —, “Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,” *SIAM review*, vol. 41, no. 2, pp. 303–332, 1999.
- [45] S. Stein, S. Sussman, T. Tomesh, C. Guinn, E. Tureci, S. F. Lin, W. Tang, J. Ang, S. Chakram, A. Li *et al.*, “Hetarch: Heterogeneous microarchitectures for superconducting quantum systems,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 539–554.
- [46] S. Stein, S. Xu, A. W. Cross, T. J. Yoder, A. Javadi-Abhari, C. Liu, K. Liu, Z. Zhou, C. Guinn, Y. Ding *et al.*, “Hetec: Architectures for heterogeneous quantum error correction codes,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025, pp. 515–528.
- [47] D. B. Tan, M. Y. Niu, and C. Gidney, “A sat scalpel for lattice surgery: Representation and synthesis of subroutines for surface-code fault-tolerant quantum computing,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, Jun. 2024, p. 325–339. [Online]. Available: <http://dx.doi.org/10.1109/ISCA59077.2024.00032>
- [48] T. Trochatos, C. Kang, A. Wang, F. T. Chong, and J. Szefer, “Trace-based reconstruction of quantum circuit dataflow in surface codes,” *arXiv preprint arXiv:2508.14533*, 2025.
- [49] Y. Ueno, T. Saito, T. Tanimoto, Y. Suzuki, Y. Tabuchi, S. Tamate, and H. Nakamura, “High-performance and scalable fault-tolerant quantum computation with lattice surgery on a 2.5 d architecture,” *arXiv preprint arXiv:2411.17519*, 2024.
- [50] K. Wang, Z. Lu, C. Zhang, G. Liu, J. Chen, Y. Wang, Y. Wu, S. Xu, X. Zhu, F. Jin *et al.*, “Demonstration of low-overhead quantum error correction codes,” *arXiv preprint arXiv:2505.09684*, 2025.
- [51] M. Wang, C. Liu, S. Stein, Y. Ding, P. Das, P. J. Nair, and A. Li, “Optimizing ftqc programs through qec transpiler and architecture codesign,” *arXiv preprint arXiv:2412.15434*, 2024.
- [52] G. Watkins, H. M. Nguyen, K. Watkins, S. Pearce, H.-K. Lau, and A. Paler, “A high performance compiler for very large scale surface code computations,” *Quantum*, vol. 8, p. 1354, 2024.
- [53] A. Wu, G. Li, H. Zhang, G. G. Guerreschi, Y. Ding, and Y. Xie, “A synthesis framework for stitching surface code with superconducting quantum devices,” in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 337–350.
- [54] Q. Xu, J. P. Bonilla Ataides, C. A. Pattison, N. Raveendran, D. Bluvstein, J. Wurtz, B. Vasić, M. D. Lukin, L. Jiang, and H. Zhou, “Constant-overhead fault-tolerant quantum computation with reconfigurable atom arrays,” *Nature Physics*, vol. 20, no. 7, pp. 1084–1090, 2024.
- [55] J. Yi, W. Ye, D. Gottesman, and Z.-W. Liu, “Complexity and order in approximate quantum error-correcting codes,” *Nature Physics*, vol. 20, no. 11, pp. 1798–1803, 2024.
- [56] K. Yin, X. Fang, Z. Chen, A. Li, D. Hayes, E. Kaur, R. Nejabati, H. Haefner, W. Campbell, E. Hudson *et al.*, “Flexion: Adaptive in-situ encoding for on-demand qec in ion trap systems,” *arXiv preprint arXiv:2504.16303*, 2025.
- [57] K. Yin, H. Zhang, X. Fang, Y. Shi, T. S. Humble, A. Li, and Y. Ding, “Qecc-synth: A layout synthesizer for quantum error correction codes on sparse architectures,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2025, pp. 876–890.
- [58] Y. Zhao, Y. Ye, H.-L. Huang, Y. Zhang, D. Wu, H. Guan, Q. Zhu, Z. Wei, T. He, S. Cao *et al.*, “Realization of an error-correcting surface code with superconducting qubits,” *Physical Review Letters*, vol. 129, no. 3, p. 030501, 2022.